



Rappel une video très claire de l'inria (avec un petit accent belge) pour présenter le rôle des systèmes d'exploitation : <https://youtu.be/SpCP2oaCx8A>

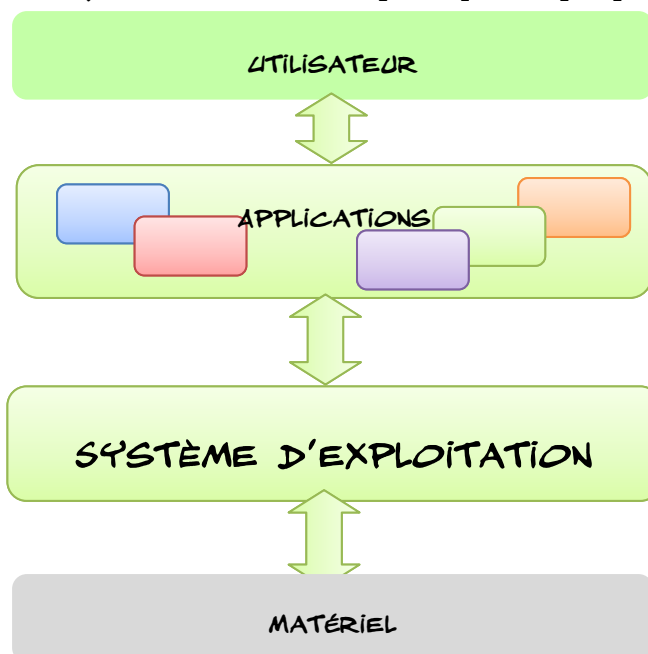
I. ORDONNANCEUR

1. SYSTÈME D'EXPLOITATION (RAPPELS)

Pour qu'un ordinateur soit capable de faire fonctionner un programme (application ou logiciel), la machine doit être en mesure d'effectuer un certain nombre d'opérations préparatoires afin d'assurer les échanges entre le processeur, la mémoire, et les ressources physiques (périphériques).

Le système d'exploitation (noté SE ou OS, abréviation du terme anglais Operating System), est chargé d'assurer l'interface entre les ressources matérielles, l'utilisateur et les applications (traitement de texte, jeu vidéo, ...).

Ainsi, lorsqu'un programme désire accéder à une ressource matérielle, il ne lui est pas nécessaire d'envoyer des informations spécifiques au périphérique, il lui suffit d'envoyer les informations au



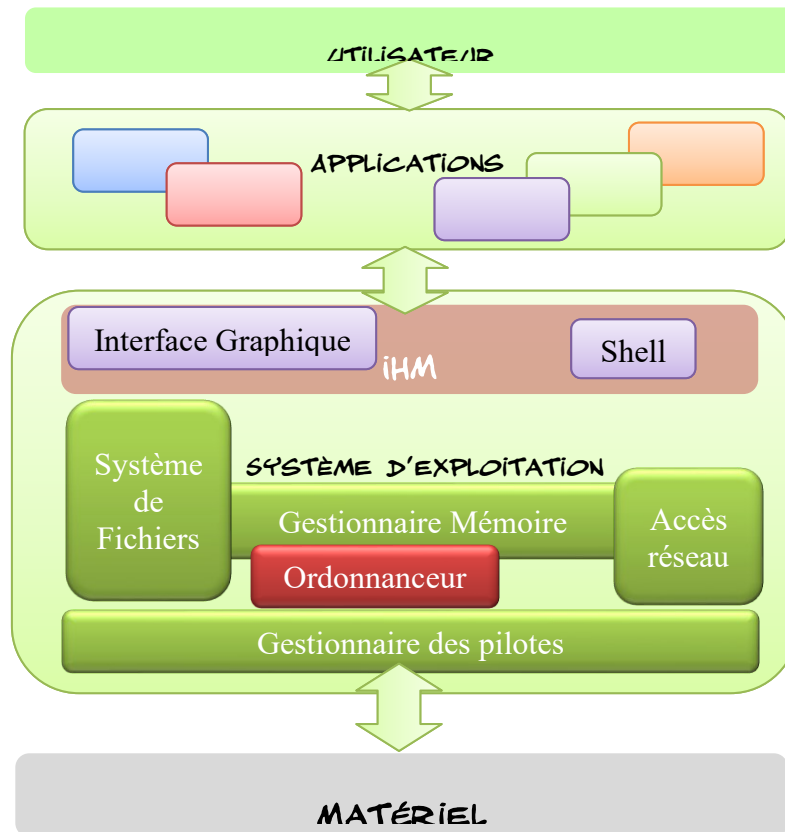
système d'exploitation, qui se charge de les transmettre au périphérique concerné via son pilote.

En l'absence de pilotes, il faudrait que chaque programme reconnaisse et prenne en compte la communication avec chaque type de périphérique !

Le système d'exploitation permet ainsi de «dissocier» les programmes applicatifs et le matériel, afin notamment.

- + de simplifier la gestion des ressources (CPU, mémoire,...)
- + d'offrir à l'utilisateur une interface homme-machine simplifiée (graphique ou textuelle)
- + permettre à l'utilisateur de s'affranchir de la complexité et des spécificités matérielles de la machine.

2. LE RÔLE DU SYSTÈME D'EXPLOITATION (RAPPELS)



II. LES PROCESSUS

1. NOTION DE PROCESSUS

Quand l'utilisateur lance l'exécution d'un programme à partir d'un OS en cliquant par exemple sur une icône, il déclenche la création d'un processus chargé d'exécuter le programme. D'autres processus peuvent être déclenchés à partir d'un processus (on dira que ce sont des processus fils).

Certains fonctionnent en permanence en arrière-plan, comme un antivirus et ceux qui s'exécutent dès le démarrage de la machine (les services sous windows ou les daemons sous linux).

On peut distinguer deux grandes catégories de processus :

ceux créés par un utilisateur,

ceux créés par le système.

EX 1 : l'interface graphique du gestionnaire des tâches sous Windows :

Gestionnaire des tâches

Fichier Options Affichage

Processus Performance Historique des applications Démarrage Utilisateurs Détails Services

Nom	Statut	100% Processeur	84% Mémoire	19% Disque	0% Réseau	3% Processeur...	Moteur de processeur graphique	Consommati...	Tendance de c...
Applications (13)									
Adobe Acrobat Reader DC (32 b...		0%	118,3 Mo	0 Mo/s	0 Mbits/s	0%		Très faible	
Client PRONOTE		0%	4,9 Mo	0 Mo/s	0 Mbits/s	0%		Très faible	
Discord (32 bits) (6)		1,8%	179,5 Mo	0,1 Mo/s	0 Mbits/s	1,4%	GPU 0 - 3D	Très faible	
Excel		0,3%	8,6 Mo	0 Mo/s	0 Mbits/s	0%		Très faible	
Explorateur Windows (3)		0,3%	38,8 Mo	0 Mo/s	0 Mbits/s	0%		Très faible	
Firefox (8)		0,5%	381,5 Mo	0 Mo/s	0 Mbits/s	0%		Très faible	
Gestionnaire des tâches		0,6%	34,2 Mo	0 Mo/s	0 Mbits/s	0%		Très faible	
Interpréteur de commandes Wi...		0%	1,7 Mo	0 Mo/s	0 Mbits/s	0%		Très faible	
iTunes (5)		0%	25,7 Mo	0 Mo/s	0 Mbits/s	0%		Très faible	
Outil Capture d'écran		0,3%	1,9 Mo	0 Mo/s	0 Mbits/s	0%		Très faible	
PyScripter (32 bits) (2)		0%	6,9 Mo	0 Mo/s	0 Mbits/s	0%		Très faible	
Thunderbird (32 bits)		0%	86,0 Mo	0 Mo/s	0 Mbits/s	0%		Très faible	
Word		0,1%	123,2 Mo	0 Mo/s	0 Mbits/s	0%	GPU 0 - 3D	Très faible	
Processus en arrière-plan (108)									
Adobe Acrobat Update Service (...)		0%	0,4 Mo	0 Mo/s	0 Mbits/s	0%		Très faible	
Adobe RdCCEF (32 bits)		0%	1,2 Mo	0 Mo/s	0 Mbits/s	0%		Très faible	

Moins de détails

Fin de tâche

2. PROGRAMME ET PROCESSUS

Un programme est une description statique d'une tâche à exécuter.

Un processus est une instance (un exemplaire) d'un programme en cours d'exécution.



Si l'on compare le code source d'un programme à une recette de cuisine listant les ingrédients et décrivant la méthode à utiliser pour la réaliser, le **processus** est alors quelqu'un qui a récupéré la recette, acheté les ingrédients et qui est en train de réaliser la recette.



De plus à un instant donné, on peut avoir plusieurs personnes dans la même cuisine. Chacun s'occupe d'une recette (la même ou pas...), chacun est sur son propre exemplaire de recette et dans un état d'avancement différent.



On peut alors :



Désigner les tâches individuellement et indépendamment du programme qu'elles exécutent (même si on a plusieurs instances du même programme).



Agir de façon individuelle sur les différentes tâches sans impacter les autres. Comme Par exemple:

- en mettre certaines en pause.
- en terminer d'autres.
- gérer les tâches en fonction de priorités (peut-être que certaines sont plus importantes que d'autres).
- ...

Exo 1. Consommation processeur

1. Quelle est l'interface Windows qui permet de visualiser les processus en cours ?
2. Sous Windows, ouvrir cette interface et un interpréteur Python.
Quelle est la consommation de l'interpréteur Python ?
3. Editer puis exécuter le programme Python suivant :
For i in range(10000) : print(i)
4. Observer et noter la consommation du processeur.
5. Ouvrir un lecteur de vidéo, par exemple VLC et visionner une vidéo.
La consommation processeur est-elle importante ?
Dans l'interface de visualisation des processus, quelles sont les colonnes impactées par cette lecture de vidéo ?

3. CREATION, DESTRUCTION D'UN PROCESSUS, CONTEXTE.

A sa création un processus se voit donc attribuer en mémoire un code exécutable ... à exécuter.

Un processus est donc un objet dynamique : son état évolue entre l'instant de sa création et l'instant de sa destruction.



A la création d'un processus, **plusieurs identifiants** lui sont affectés par le système d'exploitation :

- le **PID** (Process Identifier) qui identifie le processus lui-même (deux processus ont donc deux PID différents)
- Le **PPID** (Parent Process Identifier) permet d'identifier le processus parent d'un processus (seul le processus 0, ne possède pas de PPID).
- Le **UID** qui identifie l'utilisateur qui a démarré le processus.
- Le **GID** qui identifie le groupe de l'utilisateur qui a démarré le processus.

Le processus travaille dans un **contexte** : il s'agit de l'ensemble des informations dont il dispose pour mener à bien l'exécution du programme dont il est une instance.

On y trouve

- Les variables globales du programme exécuté,
- la pile d'exécution (qui contient les adresses des données (arguments des fonctions, variables,...), les adresses de retours d'appels de fonctions, ...),
- les valeurs stockées dans les registres,
- ou encore l'ensemble des ressources utilisées par le programme (fichiers ouverts, connexions réseau,...)

Rq1 Deux processus différents ne partagent pas leur contexte

4. ÉTATS D'UN PROCESSUS

L'exécution d'un processus est soumise à des *interruptions* : il s'agit d'un signal envoyé au processeur lorsqu'un événement se produit.

Une interruption peut être d'origine **matérielle** (clic, clavier, interruption d'horloge, fin d'écriture sur le disque dur,...) : son occurrence déclenche un programme particulier dit gestionnaire d'interruption qui détermine en fonction de l'événement déclencheur de l'interruption la suite d'instructions à exécuter.

Une interruption peut être aussi d'origine **logicielle** : elle relève alors de la décision de *l'ordonnanceur* (*scheduler* en anglais) des processus qui peut décider d'interrompre un processus et attribuer le CPU à un autre processus.



Tous les systèmes d'exploitation "modernes" (Linux, Windows, macOS, Android, iOS...) sont capables de gérer l'exécution de plusieurs processus actifs en même temps (dite concurrente). Mais pour être précis, il s'agit plutôt d'une simulation du "en même temps", par un "chacun son tour mais avec des priorités", quitte à interrompre le processus actif. Pour gérer ce "chacun son tour", le système d'exploitation attribue à chaque processus un "état".

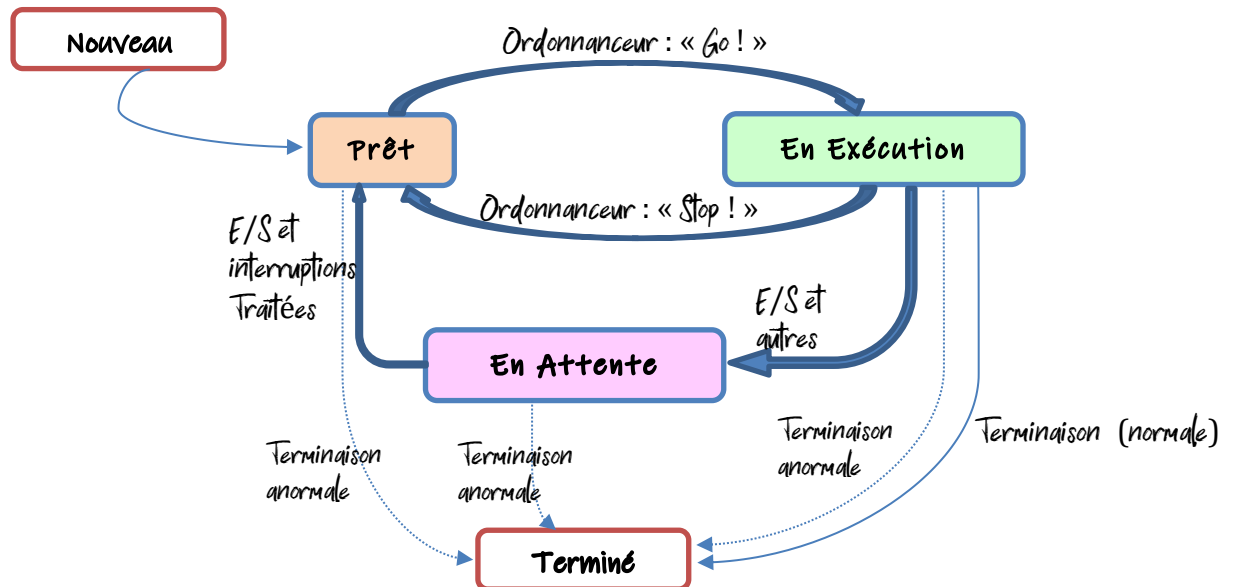
- Lorsqu'un processus est en train de s'exécuter (qu'il utilise le microprocesseur), on dira que le processus est dans l'état "en exécution".

Il ne peut y avoir **qu'un seul processus** dans l'état particulier "en exécution".

- Un processus qui se trouve dans l'état "en exécution" peut demander à accéder à une ressource pas forcément disponible instantanément (par exemple lire une donnée sur le disque dur). Le processus ne peut pas poursuivre son exécution tant qu'il n'a pas obtenu cette ressource. En attendant de recevoir cette ressource, il passe de l'état "en exécution" à l'état "en attente".
- Lorsque le processus finit par obtenir la ressource attendue, celui-ci peut potentiellement reprendre son exécution. Mais le système d'exploitation qui doit simuler le "en même temps" de plusieurs processus, peut aussi décider de donner la main (le CPU en fait) à un autre processus qui peut alors "prendre sa place" dans l'état "en exécution". Le processus qui vient alors de recevoir la ressource attendue ne va donc pas forcément pouvoir reprendre son exécution tout de suite, car pendant qu'il était dans l'état "en attente" un autre lui est passé devant... Un processus qui quitte l'état "en attente" ne repasse pas automatiquement à l'état "en exécution", il peut, en attendant que "la place se libère" passer dans l'état "prêt" (sous-entendu "j'ai obtenu ce que j'attendais, je suis prêt à reprendre mon exécution dès que la « place sera libérée »").
- Un processus est toujours créé dans l'état "prêt". Pour se terminer normalement, un processus doit obligatoirement se trouver dans l'état "en exécution". Mais il existe des cas de terminaison anormale (par exemple si le disque dur sur lequel le processus est (ou a été) en attente est déclaré défectueux, le processus se terminera anormalement à partir de l'état en attente (ou prêt)).

Diagramme des états (et des transitions)

Rq2. On fait apparaître ici des états « Nouveau » et « Terminé » qui sont des états éphémères (à usage unique, on y revient jamais).
 A l'état « nouveau », le processus s'initialise, et à l'état « Terminé », le système d'exploitation libère les ressources allouées par le processus.



Le "chef d'orchestre" qui attribue aux processus leur état "Prêt", "En Attente" ou "En Exécution" est le système d'exploitation (plus précisément l'ordonnanceur).

Rq3. A chaque processus est attribuée une **priorité** qui détermine le temps CPU que lui accorde l'ordonnanceur en fonction des exigences et des priorités de chaque processus.

5. ÉTATS ET CONTEXTES

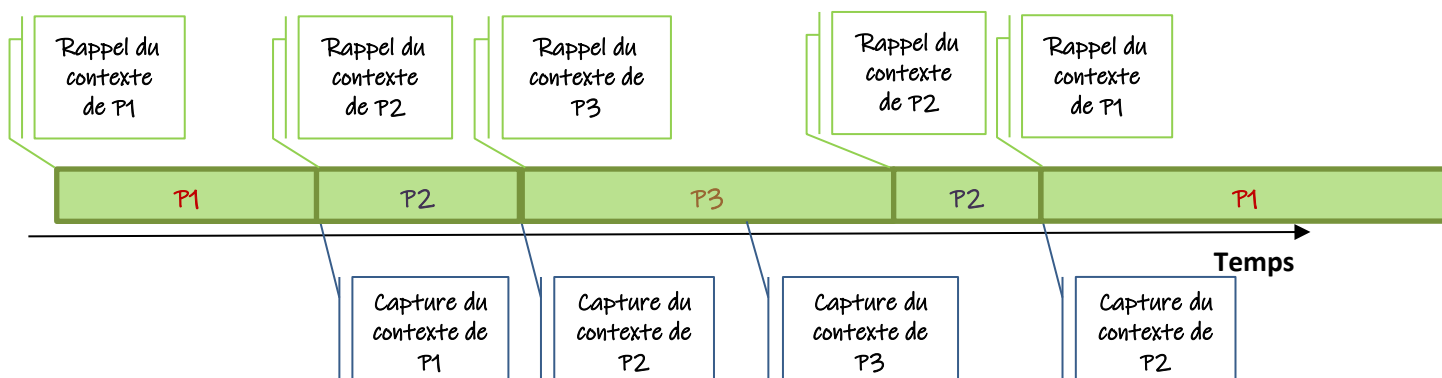
Le contexte d'un processus n'évolue que lorsqu'il est dans l'état « en exécution »

La transition d'un processus vers l'état « en exécution » entraîne la copie de son contexte des registres du processeur.

Quand un processus quitte l'état « en exécution », son contexte est capturé en mémoire (il est alors figé et sera rappelé dans les registres que lorsque le processus aura de nouveau la main).



Le schéma ci-dessous illustre l'ordonnancement de 3 processus (P1, P2, et P3) avec un seul processeur. On y voit la succession des processus « en exécution » et la commutation des contextes « actifs ».
 La **capture en mémoire** et le **rappel dans les registres** des contextes permet l'exécution d'un processus indépendamment des autres processus existants.



6. PROCESSUS SOUS WINDOWS

Exo 2.

- Ouvrir l'éditeur Bloc-notes, trouver le processus correspondant, noter son PID puis l'interrompre depuis l'interface de visualisation des processus. Recommencer l'opération. Commenter.
- Rechercher l'exécutable associé à Bloc-Notes puis à partir de l'interface de visualisation, créer une nouvelle tâche instance de Bloc-Notes. Puis une autre.

Exo 3. Moins confortable mais parfois plus efficace, le pilotage des processus à partir du shell Windows (cmd ou windows power shell à partir de la zone de recherche dans la barre des tâches) :

Invite de commandes

```
Microsoft Windows [version 10.0.19041.685]
(c) 2020 Microsoft Corporation. Tous droits réservés.

C:\Users\pierr>tasklist

Nom de l'image                PID Nom de la sessio Numéro de s Utilisation
-----
System Idle Process           0 Services                0      8 Ko
System                        4 Services                0    19 880 Ko
Registry                     100 Services              0    49 904 Ko
smss.exe                     528 Services              0     272 Ko
```

- Tester dans l'invite de commande : `tasklist`
- Pour repérer les instances de firefox par exemple : `tasklist | findstr "firefox"`
- Pour arrêter l'application firefox, il faut tuer tous les processus firefox :

On peut utiliser le PID (à récupérer avec le `tasklist`) :

```
C:\Users\pierr>taskkill /PID 11536 /T
Opération réussie : un signal de fin a été envoyé au processus de PID 11536,
processus enfant de PID 5060.
```

option `/T` : pour tuer tous les processus enfants

Tester.

ou encore :

```
C:\Users\pierr>taskkill /f /PID 14152 /f
Opération réussie : le processus de PID 13960 (processus enfant de PID 13216) a été
arrêté.
Opération réussie : le processus de PID 12008 (processus enfant de PID 14152) a été
arrêté.
Opération réussie : le processus de PID 14028 (processus enfant de PID 14152) a été
arrêté.
Opération réussie : le processus de PID 13676 (processus enfant de PID 14152) a été
arrêté.
```

```
C:\Users\pierr>taskkill /PID 11536 /T
Opération réussie : un signal de fin a été envoyé au processus de PID 11536,
processus enfant de PID 5060.
```

utilité de l'option `/f` :

On peut aussi utiliser le nom de l'exécutable :

```
C:\Users\pierr>taskkill /f /im firefox.exe
Opération réussie : le processus "firefox.exe" de PID 6264 a été arrêté.
Opération réussie : le processus "firefox.exe" de PID 20236 a été arrêté.
Opération réussie : le processus "firefox.exe" de PID 19360 a été arrêté.
```

Tester.

III. LES RESSOURCES

1. PARTAGE DES RESSOURCES

Une ressource est un objet (matériel, fichier, donnée...), géré par le système et dont le système peut concéder temporairement le droit d'utilisation à un, ou plusieurs processus.



L'utilisation de ressources se fait en trois étapes:

- Demande de la ressource ;
- Utilisation de la ressource lorsqu'elle est allouée ;
- Libération de la ressource.



Mais lorsqu'un processus demande la ressource au système d'exploitation, plusieurs issues sont possibles :

- Acceptation et allocation immédiate
- Blocage en attente de disponibilité de la ressource
- Refus (droits insuffisants, ressource inexistante, etc.)

Le partage de ressources entre processus peut donner lieu à des dysfonctionnements (à éviter, donc) :

- Privation (ou famine) : un processus attend indéfiniment une ressource,
- Congestion : des allocations mal contrôlées conduisent à une surcharge du système, et à une dégradation de ses performances,
- Interblocage (*deadlock* traduit parfois en « étreinte fatale ») : des processus se trouvent bloqués mutuellement, et indéfiniment.

2. INTERBLOCAGE (DEADLOCK)



Les interblocages sont des situations que l'on rencontre dans la vie quotidienne.

Un exemple est celui du carrefour avec priorité à droite où chaque véhicule est bloqué car il doit laisser le passage au véhicule à sa droite.



En informatique, l'interblocage peut se produire lorsque des processus concurrents s'attendent mutuellement. Les processus bloqués dans cet état le sont définitivement. Ce scénario peut se produire dans un environnement où des ressources sont partagées entre plusieurs processus et l'un d'entre eux détient indéfiniment une ressource nécessaire pour l'autre.



Cette situation d'interblocage a été théorisée par l'informaticien Edward Coffman (1934-) qui a énoncé quatre conditions (appelées conditions de Coffman) menant à l'interblocage :

1. **Exclusion mutuelle** : au moins une des ressources du système doit être en accès exclusif (un seul processus est autorisé à y accéder au moment du blocage)
2. **Rétention des ressources** : un processus détient (et finalement *retient*) au moins une ressource et demande une autre ressource détenue par un autre processus
3. **Non préemption** : Seul le détenteur d'une ressource peut la libérer et il est impossible de *préempter* la ressource (c'est-à-dire « la *réquisitionner* »)
4. **Attente circulaire** : Chaque processus attend une ressource détenue par un autre processus. P_1 attend une ressource détenue par P_2 qui à son tour attend une ressource détenue par P_3 etc... et le dernier attend une ressource détenue par P_1 .

Exo 4. Dans la situation du carrefour, dans laquelle chaque véhicule cherche à traverser, identifier précisément les processus et les ressources partagées.
Vérifier alors que les conditions de Coffman sont bien vérifiées.

Exo 5. Deadlock à l'Olympia.

Constater le blocage à l'Olympia et vérifier les conditions de Coffman dans le scénario suivant :

- a. Alice, pas trop sûre de vouloir aller toute seule à l'Olympia, prend une option annulable sur la place 1;
- b. Bob, qui ne connaît pas Alice, et pas trop sûr non plus de vouloir y aller tout seul, prend une option annulable sur la place 2 (voisine de la 1);
- c. Alice, qui envisage finalement de sortir en couple (pas avec Bob qu'elle ne connaît pas plus qu'il ne la connaît, mais plutôt avec Antoine), demande la place 2 ; on lui répond
 - « il y a une option sur la place 2. Un dénommé Bob, vous connaissez ?
 - non.
 - bon, de toute façon, il n'avait pas l'air trop décidé. Cette place a des chances de se libérer, on vous met en attente dessus ?
 - oui, c'est cool »
- d. Bob qui envisage finalement de sortir en couple (pas avec Alice, et pas non plus avec Antoine, et plutôt avec Babette), demande la place 1 ; on lui répond
 - « il y a une option sur la place 1. Une dénommée Alice, vous connaissez ?
 - non.
 - je m'en doutais. De toute façon, elle n'avait pas l'air trop décidée. Cette place a des chances de se libérer, on vous met en attente dessus ?
 - oui, c'est cool. »
- e. Alice et Bob sont très patients. Antoine et Babette sont partis boire un café, mais pas ensemble.

Ex 2 : Un exemple générique avec deux processus et deux ressources :

Supposons un système possédant les ressources 1 et 2, ayant les processus A et B en exécution. Une façon de provoquer un interblocage est la suivante :

- Le processus A réserve et obtient la ressource 1 ;
- Le processus B réserve et obtient la ressource 2 ;
- Le processus A demande la ressource 2, et est mis en attente ;
- Le processus B demande la ressource 1, et est mis en attente ;

Interblocage !

[SY Interblocage.docx](#)

Exo 6. Ecrire un scénario d'interblocage (vraisemblable, c'est mieux) dans la vie courante.

Exo 7. Un interblocage à trois processus et 3 ressources :

On considère 3 Processus P1, P2 et P3 et 3 ressources R1, R2 et R3 tels que :

- P1 : demande R1, demande R2, libère R1, libère R2
- P2 : demande R2, demande R3, libère R2, libère R3
- P3 : demande R3, demande R1, libère R3, libère R1

1. Si les processus sont exécutés l'un après l'autre, d'abord P1 puis P2 et enfin P3, y-a-t-il interblocage ?
2. Décrire une exécution des 3 processus qui conduit à une situation d'interblocage

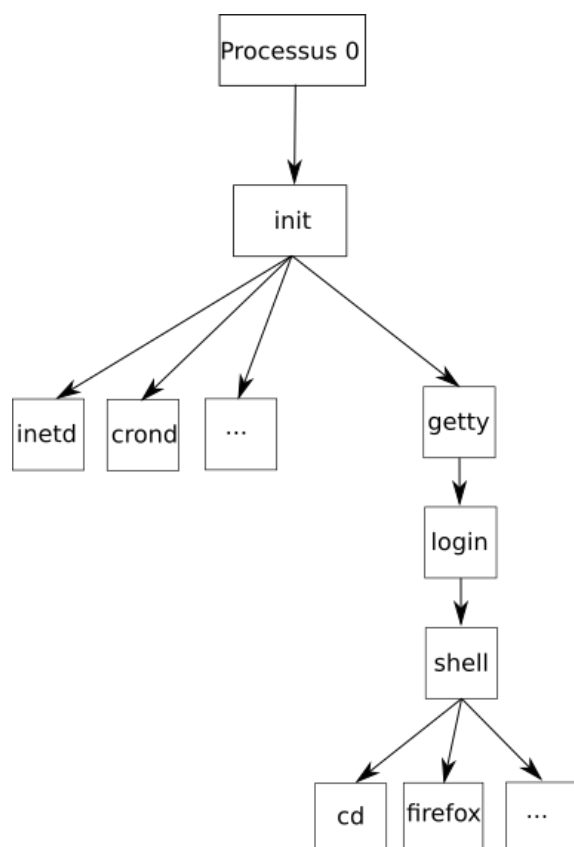
IV. Exos

Pour manipuler sous Linux, on utilisera l'émulateur de système mis au point par Fabrice Bellard accessible à l'adresse :

<https://bellard.org/jslinux/vmps -ef.html?url=https://bellard.org/jslinux/buildroot-x86.cfg>

1. PROCESSUS SOUS LINUX

Un processus peut créer un ou plusieurs processus à l'aide d'une commande système ("fork" sous Linux). On peut modéliser ces relations père/fils par une structure arborescente.



Sous un système d'exploitation comme Linux, au moment du démarrage de l'ordinateur, un tout premier processus (appelé processus 0 ou encore Swapper) est créé à partir de "rien" (il n'est le fils d'aucun processus). Ensuite, ce processus 0 crée un processus souvent appelé "init" ("init" est donc le fils du processus 0). À partir de "init", les processus nécessaires au bon fonctionnement du système sont créés (par exemple les processus "crond", "inetd", "getty",...). Puis d'autres processus sont créés à partir des fils de "init"... ; les PID sont alors incrémentés.



Ce sont ces processus ainsi que les processus applicatifs qu'il est ensuite possible de visualiser à travers un terminal de commande.

Exo 8. Visualiser les processus



Il est possible de les visualiser grâce à la commande `ps -ef`.

Pour un affichage page par page, utilisez `ps -ef | more`

1. Quel est le PID du processus **init** ?
2. Quel est le PPID de **init** ?
3. **init** possède-t-il un frère ?
4. Citer quelques descendants directs de **init**

Exo 9. Inspecter les processus en temps réel

Une commande utile à connaître sous Linux pour inspecter les processus en temps réel : la commande `top`.

Provide a view of process activity in real time.
Read the status of all processes from /proc each SECONDS
and display a screenful of them.

Keys:

N/M/P/T: sort by pid/mem/cpu/time

R: reverse sort

Q, ^C: exit

Options:

```
-b      Batch mode
-n N    Exit after N iterations
-d SEC  Delay between updates
```



Lancer cette commande dans un terminal.

On obtient quelque chose comme ci-contre.

L'affichage se rafraîchit en temps réel contrairement à `ps` qui fait un instantané.

```
top - 15:07:07 up 6:47, 1 user, load average: 0,08, 0,15, 0,15
Tâches: 355 total, 1 en cours, 354 en veille, 0 arrêté, 0 zombie
%Cpu(s): 0,5 ut, 0,2 sy, 0,0 ni, 99,4 id, 0,0 wa, 0,0 hi, 0,0 si, 0,0 st
MiB Mem : 7943,8 total, 1954,2 libr, 3571,0 util, 2418,6 temp/cache
MiB Ech: 8192,0 total, 8182,2 libr, 9,8 util, 3898,7 dispo Mem
```

PID	UTIL.	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TEMPS+	COM.
13266		20	0	2401412	636112	91908	S	0,0	7,8	2:22.27	prusa-slicer
16853		20	0	3976324	521448	242448	S	0,3	6,4	14:34.10	firefox
35838		20	0	2723516	280100	203844	S	0,0	3,4	0:12.40	Web Content
22685		20	0	1003148	261068	103364	S	0,0	3,2	6:49.57	codium
16912		20	0	2805020	238128	142204	S	0,0	2,9	1:52.82	Web Content
17956		20	0	2609260	212680	107464	S	0,0	2,6	1:56.15	Web Content
27297		20	0	2558816	193960	112772	S	0,0	2,4	0:18.59	Web Content
17477		20	0	2695212	193904	117896	S	0,0	2,4	0:33.98	Web Content
23837		20	0	2594596	193492	125000	S	0,0	2,4	0:41.62	Web Content
18025		20	0	2534712	187416	105008	S	0,7	2,3	0:48.02	Web Content
16966		20	0	26,5g	183920	105228	S	0,0	2,3	1:04.27	WebExtension
28860		20	0	2585752	180304	125588	S	0,0	2,2	1:03.22	Web Content
7832	gn	20	0	4083444	173200	93284	S	0,0	2,1	0:10.78	gnome-shell
9731	root	20	0	508852	139752	108040	S	1,0	1,7	6:52.23	Xorg
9975		20	0	2099160	137236	79972	S	0,0	1,7	0:33.29	caja
22657		20	0	373364	133548	92252	S	0,0	1,6	2:40.30	codium
22626		20	0	669516	130884	92324	S	0,0	1,6	1:17.23	codium

L'application est plus riche qu'il n'y paraît et nécessite un peu de temps pour explorer toutes les options d'affichage, prises en compte en temps réel.

Celles-ci s'activent par des raccourcis clavier. Par exemple :

- `h` : affiche l'aide
(non implémenté sur l'émulateur)
- `N` : trie la liste par ordre décroissant des PID.
- `M` : trie la liste par ordre décroissant d'occupation mémoire. Pratique pour repérer les processus trop gourmands.
- `P` : trie la liste par ordre décroissant d'occupation processeur.
- `R` : inverse l'ordre de tri actif.
- `i` : filtre les processus inactifs. Cela ne montre que ceux qui travaillent réellement.
(non implémenté sur l'émulateur)
- `k` : permet de tuer un processus - à condition d'en être le propriétaire. Essayez de tuer `init` ...
(non implémenté sur l'émulateur)
- `v` : permet d'avoir la vue arborescente sur les processus.
(non implémenté sur l'émulateur)
- `q` : permet de quitter `top`

1. Tester les options de la commande `top`
2. Tracer l'arborescence (liaisons père/fils) des principaux process visibles.
3. Dans le cas où on n'utilise pas l'émulateur :
Observer l'évolution des informations fournies par la commande `top` quand on ouvre ou supprime un onglet dans votre navigateur web.

Exo 10. Terminer un processus

1. Lancer 3 commandes du type « `top &` » : le caractère `&` permet de lancer un processus (instance du programme `top`) en arrière-plan : le programme `top` ne s'exécute pas dans le contexte du shell mais dans celui d'un nouveau processus ; le shell rend alors immédiatement la main.
2. Repérez les *PID* des trois processus exécutant `top` puis tuez-en un à l'aide de la commande `kill`.



Pour tuer un processus, on lui envoie un signal de terminaison. On en utilise principalement 2 :

- `SIGTERM` (15) : demande la terminaison d'un processus. Cela permet au processus de se terminer proprement en libérant les ressources allouées.
- `SIGKILL` (9) : demande la terminaison immédiate et inconditionnelle d'un processus. C'est une terminaison violente à n'appliquer que sur les processus récalcitrants qui ne répondent pas au signal `SIGTERM`.



Pour terminer `top` proprement, vous lui enverrez donc un signal `SIGTERM` en tapant le numéro 15 : commande shell `kill -15 PID` où *PID* désigne le numéro du processus à quitter proprement.

Si ce dernier est planté et ne réagit pas à ce signal, alors vous pouvez vous en débarrasser en tapant `kill -9 PID`.

3. Repérer les *PID* restant à l'aide de la commande `ps` ou `top`.

Exo 11. (Bac 2021)

Avec l'option `-eo pid,ppid,stat,command`, la commande `ps` affiche, pour chaque processus, dans l'ordre :

- l'identifiant du processus *PID* (*process identifier*),
- le *PPID* (*parent process identifier*),
- l'état *STAT* et
- le nom de la commande à l'origine du processus (le programme instancié par le processus)

Les valeurs du champ *STAT* indiquent l'état des processus :

- **R : processus en cours d'exécution (*Running*)**
- **S : processus en attente (*sleeping*)**

Sur un ordinateur, on exécute la commande `ps -eo pid,ppid,stat,comm` et on obtient un affichage dont on donne ci-dessous un extrait.

```
$ ps -eo pid,ppid,stat,command
PID PPID STAT COMMAND
1 0 Ss /sbin/init
....
1912 1908 Ss Bash
2014 1912 Ss Bash
1920 1747 Sl Gedit
2013 1912 Ss Bash
2091 1593 Sl /usr/lib/firefox/firefox
5437 1912 Sl python programme1.py
5440 2013 R python programme2.py
5450 1912 R+ ps -eo pid,ppid,stat,command
```

À l'aide de cet affichage, répondre aux questions ci-dessous.

1. Quel est le nom de la première commande exécutée par le système d'exploitation lors du démarrage ?
2. Quels sont les identifiants des processus actifs sur cet ordinateur au moment de l'appel de la commande `ps` ? Justifier la réponse.
3. Depuis quelle application a-t-on exécuté la commande `ps` ?
Donner les autres commandes qui ont été exécutées à partir de cette application.
4. Expliquer l'ordre dans lequel les deux commandes `python programme1.py` et `python programme2.py` ont été exécutées.
5. Peut-on prédire que l'une des deux commandes `python programme1.py` et `python programme2.py` finira avant l'autre ?